ACCELERATING LLM INFERENCE VIA SET BLOCK DECODING

Yuhang Hu [1] **Miaoyang Ye** [1]

ABSTRACT

Standard LLM inference suffers from a memory-bandwidth bottleneck due to sequential next-token prediction, which limits real-world deployment. To address this, we first implement Set Block Decoding (SBD) in Pytorch, utilizing a hybrid attention mechanism and joint training objective to prepare the model for parallel, non-contiguous token generation at inference time. We validate that SBD training is stable and largely preserves autoregressive performance on GPT-2 small. We then convert the fine-tuned weights to MiniTorch format and implement the full inference pipeline from scratch, including the Entropy Bounded (EB) Sampler and KV-cache. End-to-end benchmarking demonstrates up to $3.56\times$ fewer forward passes and $2.67\times$ wall-clock speedup. Code and models are available at https://github.com/hailey-huyuhang/llmsys-project-set-block-decoding.

1 INTRODUCTION / MOTIVATION

Standard LLM inference is severely hindered by the sequential nature of Next-Token Prediction (NTP), which requires a full model forward pass to generate each individual token. This creates a massive memory-bandwidth bottleneck, as the GPU must repeatedly read the entire model's weights and KV-cache while performing very few floating-point operations (FLOPs) per token.

The primary system challenge is to break this sequential dependency without altering the underlying Transformer architecture or adding auxiliary models. We must enable the parallel sampling of non-contiguous future tokens while maintaining compatibility with exact KV-caching and ensuring that the model does not lose its foundational autoregressive reasoning performance during the acceleration process.

In this work, we reimplement SBD (Gat et al., 2025) to address these challenges. Our contributions span training, inference, and benchmarking. We fine-tune GPT-2 small on WikiText-2 using a hybrid attention mask and a joint NTP+MATP loss, and validate that SBD training is stable and preserves autoregressive performance. We then convert the trained weights to MiniTorch format and implement the full SBD inference pipeline from scratch, including the hybrid inference mask, KV-cache, and Entropy Bounded (EB) Sampler. Finally, we benchmark the system end-to-end across block sizes $k \in \{4, 8, 16\}$ and entropy thresholds $\gamma \in \{3.5, 6.0, 10.0, 16.0\}$, achieving up to $3.56\times$ fewer forward passes and $2.67\times$ wall-clock speedup at $k = 16$, $\gamma = 16.0$.

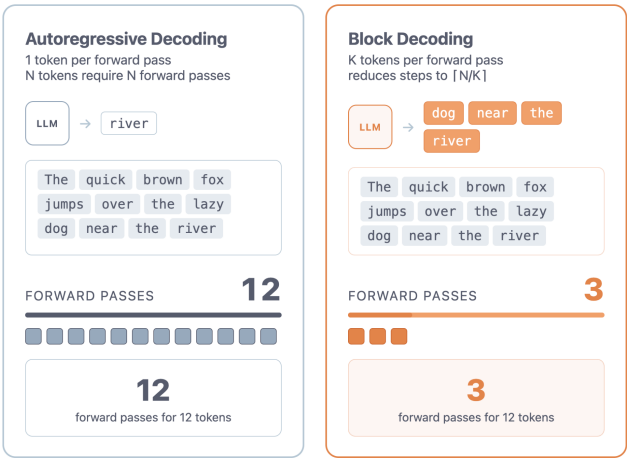


Figure 1. From One-by-one Token Generation to Parallel Block Generation.

2 RELATED WORK / BACKGROUND

Reducing the memory-bandwidth bottleneck of sequential NTP has been a central focus of recent LLM inference research. One central approach for accelerating this process is speculative decoding, which utilizes a fast draft model to generate candidate tokens for parallel verification by a slower target model (Leviathan et al., 2023). Alternatively, blockwise parallel decoding or multi-token prediction frameworks attach additional decoding heads to the base model itself to predict multiple independent tokens into the future (Stern et al., 2018; Cai et al., 2024). However, in both cases, the target model is generally restricted to verifying and accepting only a consecutive subsequence of these tokens,

[*]Equal contribution [1]Carnegie Mellon University, Pittsburgh, PA, USA. Correspondence to: <>.

Proceedings of the 5th MLSys Conference, Santa Clara, CA, USA, 2024. Copyright 2024 by the author(s).

and these methods often introduce significant architectural modifications or system complexity. SBD requires no draft model or extra heads, only fine-tuning the existing model with a joint loss.

3 METHODOLOGY

3.1 Overall Pipeline

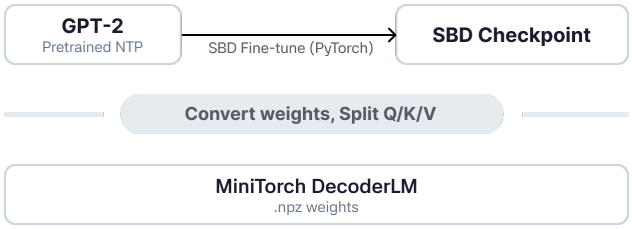


Figure 2. Overall Pipeline

3.2 SBD Training Formulation

We address these questions by implementing Set Block Decoding (SBD) (Gat et al., 2025), which enables parallel token prediction through a hybrid attention mechanism and a joint training objective.

- **Hybrid Transformer Architecture**: We developed a dual-purpose attention mask using `build_sbd_train_mask_dense()`: standard causal attention for historical tokens and bidirectional attention for future prediction blocks, as illustrated in Figure 3. Additionally, masked tokens are prevented from attending to future clean tokens, ensuring consistency with autoregressive constraints.

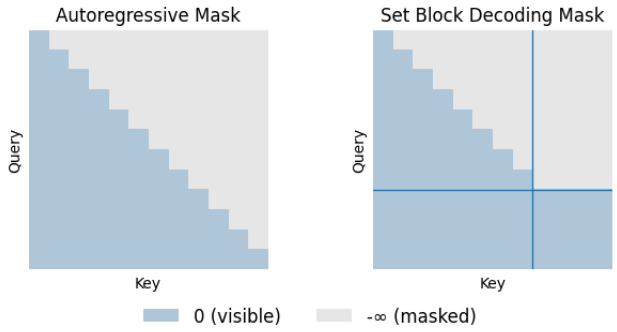


Figure 3. Comparison of Causal and Block-Structured Attention Masks

- **Joint Training Loss**: We implemented the joint loss function $\mathcal{L}(x, \hat{x}; \theta) = \mathcal{L}_{\text{NTP}}(x; \theta) + \mathcal{L}_{\text{MATP}}(\hat{x}; \theta)$, which integrates Next-Token Prediction (NTP) and Masked-Token Prediction (MATP) to ensure parallel generation abilities without degrading foundational autoregressive performance. The two objectives are equally weighted via a single mean cross-entropy reduction over concatenated valid tokens. The masking rate $\tau \sim \text{Uniform}(0, 1)$ is sampled independently per sequence, following Algorithm 1 of (Gat et al., 2025). Block length k is sampled uniformly from $[2, 16]$ per step rather than fixed, so the model learns to decode blocks of any size at inference time.

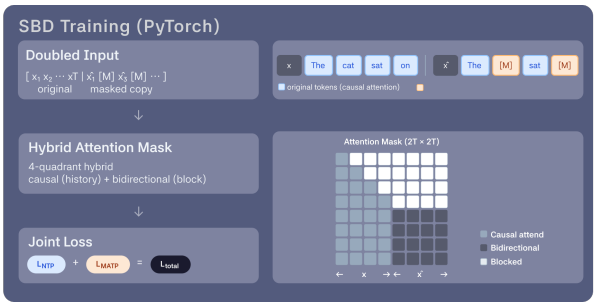


Figure 4. Training Pipeline

- **Shared Positional Embeddings**: Both halves of the doubled input share position indices $0, \ldots, T-1$ via `position_ids = torch.arange(T).repeat(1, 2)`, matching the paper's requirement that each token location receives the same positional encoding regardless of which half it appears in.

3.3 SBD Inference in MiniTorch

We implement the SBD inference pipeline in MiniTorch, covering weight conversion, model modification, attention masking, the EB Sampler, and a KV-cache decoding loop. The key challenge is bridging the gap between the PyTorch training environment and MiniTorch, since the two frameworks use incompatible weight formats and operator sets. We address this through a standalone conversion script and a set of targeted modifications to the existing MiniTorch DecoderLM.

- **Weight Conversion**: We implement `convert_weights.py` to bridge the HuggingFace and MiniTorch formats. The key challenge is that GPT-2 stores Q, K, V as a single fused projection, while MiniTorch `DecoderLM` expects three separate weight matrices. We slice the fused tensor and save all arrays as a `.npz` file.

- **MiniTorch Model Modification**: We add an optional `mask` parameter to `MultiHeadAttention`, `TransformerLayer`, and `DecoderLM`. When

`mask=None` the behaviour is identical to the original implementation; when provided, the mask is added to the attention logits before softmax, injecting the SBD attention pattern at inference time without any architectural change.

- **Inference Mask**: `build_sbd_inference_mask` constructs a $(1, H, T_{\text{full}}, T_{\text{full}})$ additive mask where positions before `causal_point` attend causally and positions in the current block attend to all positions (Figure 10 of the paper). Only the block rows are passed to the model per forward call, keeping the shape consistent with `past_kvs`.
- **EB Sampler**: `entropy_bounded_sample` computes per-position entropy for all masked positions, sorts ascending, and unmasks the largest prefix s such that the cumulative entropy of positions $1, \ldots, s-1$ stays within γ. A larger γ allows more positions to be unmasked per forward pass, reducing the total number of forward passes needed to complete a block.
- **KV-Cache and Decoding Loop**: `generate_ntp` prefills the prompt once and decodes one token at a time reusing cached K/V. `generate_sbd` prefills once, then calls `sample_block` for each block. Within `sample_block`, the EB Sampler iterates until all masked positions are revealed; a final forward pass appends the completed block's K/V to the cache for the next block.

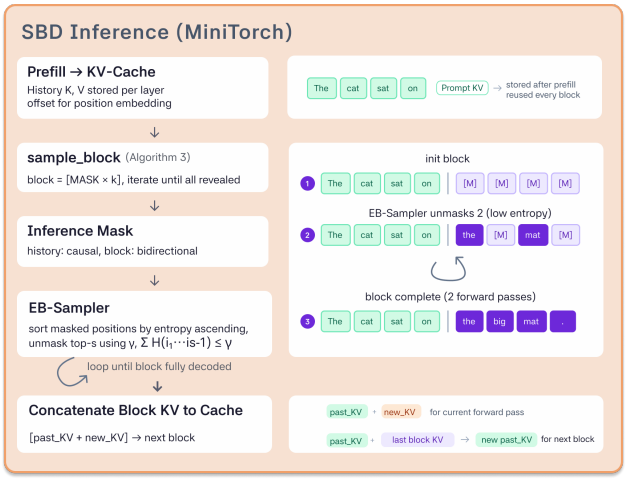


Figure 5. SBD Inference Pipeline

4 EXPERIMENTS

4.1 Experimental Setup

We use GPT-2 small (117M parameters) as the base model, fine-tuned on WikiText-2. The GPT-2 tokenizer is extended with a `[MASK]` token for masked prediction. Sequences are chunked to $T = 128$ tokens. We train with AdamW (lr = 3×10^{-4}, batch size 8) for 34,000 steps. For SBD training, block length k is sampled uniformly from $[2, 16]$ per step, and the input is doubled by concatenating the original sequence with a randomly masked copy, yielding a $2T$-length input. We implement a 4D attention mask with hybrid causal/bidirectional structure. All experiments are conducted on a single NVIDIA Tesla V100-SXM2-32GB GPU.

4.2 Baseline Training

We first validate the standard next-token prediction (NTP) pipeline by training the model in autoregressive mode for 34,000 steps. The cross-entropy loss decreases from 3.99 at step 0 to approximately 0.63 at step 34,000 (Figure 6), confirming that the training loop and data pipeline function correctly. This serves as the reference for all subsequent SBD comparisons.

4.3 SBD Training

We train SBD for 34,000 steps, logging the total loss $\mathcal{L}_{\text{total}}$, the NTP component $\mathcal{L}_{\text{NTP}}$, and the MATP component $\mathcal{L}_{\text{MATP}}$ at each step. Results are shown in Figures 6, 7, and 8. During training, we sample the block size k uniformly from $[2, 16]$ to cover a range of prediction spans.

SBD training preserves NTP performance. The NTP component converges to approximately 0.76 at 34,000 steps, slightly above the baseline value of 0.63. This small gap is consistent with the original paper's observation that adding the MATP objective results in only a slightly higher NTP loss compared to standard NTP training.

The model learns masked prediction and continues to converge. $\mathcal{L}_{\text{MATP}}$ decreases from 9.3 at step 0 to approximately 3.1 at step 34,000, a reduction of 67%, with no sign of plateauing. $\mathcal{L}_{\text{total}}$ decreases steadily from 6.2 to approximately 2.1 with no NaN values or loss explosions observed, confirming that joint training is numerically stable.



Figure 6. NTP loss: SBD NTP component matches baseline, confirming no degradation.

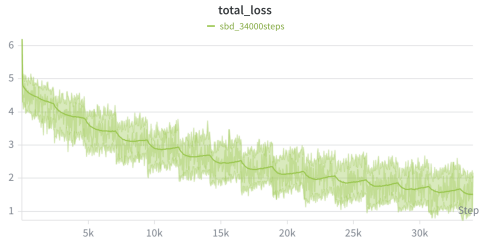


Figure 7. Total loss: combined SBD loss converges.

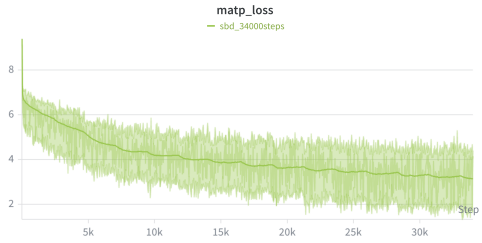


Figure 8. MATP loss with variable block size ($k \sim \text{Uniform}[2, 16]$). The loss decreases from 9.3 to approximately 3.1.

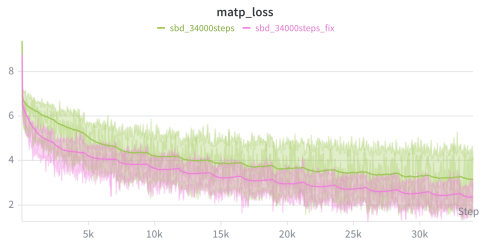


Figure 9. MATP loss comparison between fixed block size ($k = 4$) and variable block size ($k \sim \text{Uniform}[2, 16]$) training. Variable block size yields lower MATP loss at the same number of steps.

4.4 Inference Benchmark

We benchmark NTP and SBD decoding across three block sizes ($k \in \{4, 8, 16\}$) and four entropy thresholds ($\gamma \in \{3.5, 6.0, 10.0, 16.0\}$) on a single prompt of 32 new tokens. We report NFE speedup, wall-clock speedup, and perplexity (PPL) computed by re-scoring the generated output with the NTP model. Results are summarized in Table 1.

Figure 10 and 11 visualize NFE/token, wall-clock speedup, and the quality–speed tradeoff across configurations. Wall-clock speedup is consistently lower than NFE speedup because of KV-cache concatenation overhead in our NumPy implementation. The configuration with block size 16 and $\gamma = 10$ achieves the best balance of speed and output quality, with 2.30x wall-clock speedup and PPL of 96.5.

Method	k	γ	NFE/tok	NFE↑	Wall↑	PPL
NTP	–	–	1.000	1.00×	1.00×	1.67
SBD	4	3.5	0.594	1.68×	1.15×	149.1
SBD	4	6.0	0.500	2.00×	1.26×	453.2
SBD	4	10.0	0.500	2.00×	1.28×	1289.8
SBD	4	16.0	0.281	3.56×	1.78×	2738.5
SBD	8	3.5	0.531	1.88×	1.43×	144.2
SBD	8	6.0	0.438	2.29×	1.67×	1301.7
SBD	8	10.0	0.375	2.67×	1.90×	734.5
SBD	8	16.0	0.281	3.56×	2.32×	3105.8
SBD	16	3.5	0.562	1.78×	1.52×	315.0
SBD	16	6.0	0.469	2.13×	1.80×	107.6
SBD	16	10.0	0.344	2.91×	2.30×	**96.5**
SBD	16	16.0	0.281	3.56×	2.67×	4954.6

Table 1. NFE speedup, wall-clock speedup, and PPL across block sizes and entropy thresholds. PPL is computed by re-scoring generated outputs with the NTP model. The highlighted row achieves the best quality-speed tradeoff.

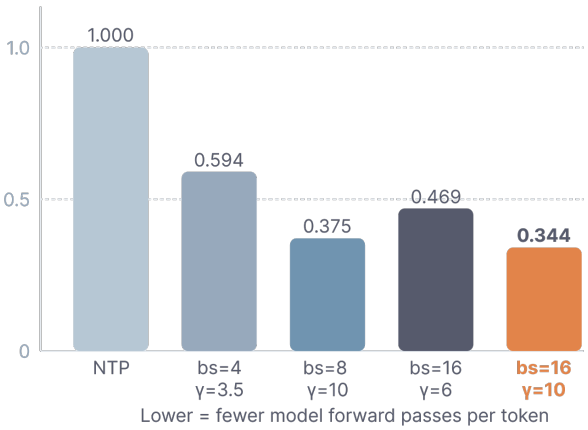


Figure 10. NFE per Token

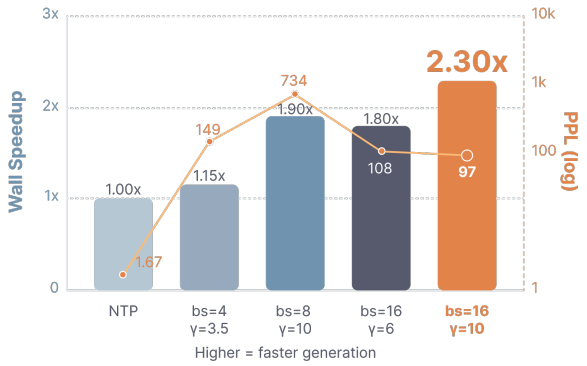


Figure 11. Wall-clock Speedup & PPL

5 Analysis/Ablations

5.1 Effect of Training Steps

Benchmarks for the 5,000-step and 34,000-step fixed block size models produce nearly identical results, with NFE and wall-clock speedup differing by no more than 0.02× under the same configuration. This suggests that on WikiText-2, the prediction entropy distribution of GPT-2 small stabilizes after roughly 5,000 steps and changes little with further training. We attribute this to the small size of the dataset: with only approximately 2M tokens, the model approaches overfitting within a few epochs, and additional training steps primarily re-expose the model to the same data rather than improving MATP capability.

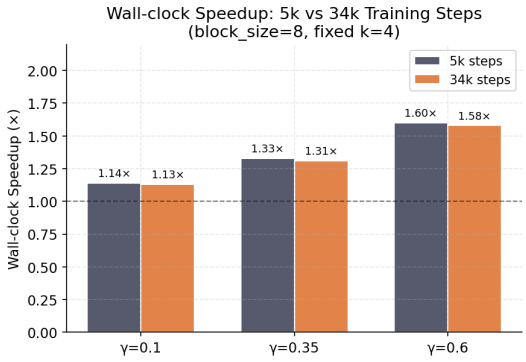


Figure 12. Wall-clock speedup at 5,000 and 34,000 training steps under fixed block size ($k = 8$) across γ values.

5.2 Effect of KV-Cache

Without KV-cache, we benchmark a 5,000-step fixed block size model with $k = 4$ and $\gamma \in \{0.1, 0.35, 0.6\}$. NFE speedup is only 1.07× across all γ values, because with $k = 4$ and small γ, the EB-Sampler unmasks only one token per step, leaving almost no room for parallel decoding. Beyond the limited NFE reduction, the absence of KV-cache means prompt tokens are recomputed in every forward pass, so even the small NFE saving does not translate to meaningful wall-clock speedup. The original paper states that SBD maintains compatibility with exact KV-caching, therefore we implement a numpy-based KV-cache in MiniTorch, managing historical K and V via `np.concatenate` and correcting position embeddings through an `offset` parameter. After adding KV-cache, NFE speedup reaches up to 2.67× at $\gamma = 10.0$ with $k = 8$, as shown in Figure 13. This confirms that KV-cache is a necessary condition for SBD acceleration.

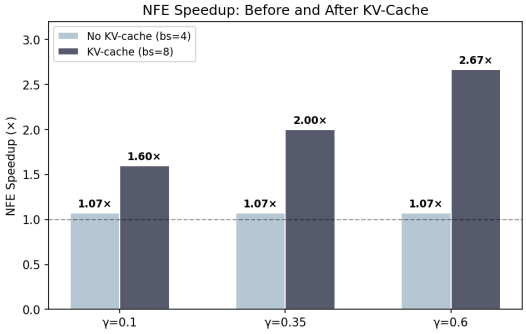


Figure 13. NFE speedup with and without KV-cache

5.3 Effect of Variable Block Size

Compared to fixed block length training with $k = 4$, sampling k uniformly from $[2, 16]$ results in lower MATP loss at the same number of training steps, as shown in Figure 9. This suggests that exposing the model to different prediction spans during training improves convergence.

5.4 Entropy Distribution and Block Size

The entropy threshold γ needs to be calibrated to the model and training setup. The original paper reports much smaller γ values than the ones used in our benchmark. In contrast, our GPT-2 small model produces block-token entropies between 3.16 and 7.18 nats in our probing experiments. When we apply the paper's smaller γ values directly, the EB-Sampler often unmasks only one token per step, yielding little or no speedup. This motivates sweeping a larger range of $\gamma \in \{3.5, 6.0, 10.0, 16.0\}$ for our model.

One possible explanation is the difference in model and training scale. The paper evaluates larger models, such as Llama-3.1 8B and Qwen-3 8B, while our experiments use GPT-2 small trained on WikiText-2. A smaller model may be less confident when predicting multiple future tokens in parallel, so larger γ values are needed to trigger multi-token unmasking. Since we do not evaluate the paper's models under the same setup, this should be viewed as a possible explanation rather than a direct comparison.

Block size also has a significant effect. With $k = 4$, even a large γ can save at most three forward passes per block, leaving limited room for speedup. Larger block sizes of 8 and 16 provide more parallel decoding capacity and achieve higher NFE speedup under the same γ.

5.5 Analysis of Benchmark Results

NFE and wall-clock gap. As shown in Figure 14, wall-clock speedup is approximately 65% to 75% of NFE speedup across all settings. Beyond numpy concatenation overhead, per-forward data movement between MiniTorch tensors and numpy arrays contributes additional latency that is disproportionately large for short sequences. Replacing numpy-based KV-cache with a custom CUDA kernel would substantially reduce this gap.

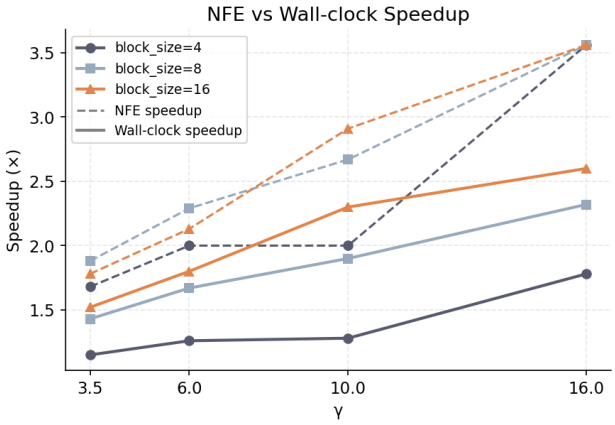


Figure 14. NFE speedup

Generation quality degrades as γ increases. As shown in Table 1 and Figure 15, PPL increases with γ in most settings, consistent with the expected speed-accuracy tradeoff. At low γ values such as 3.5, PPL remains in the range of 100–315, while at $\gamma = 16.0$ it rises to 2738–4954. The NTP baseline achieves a PPL of 1.67, which is expected since we score the model's own greedy outputs.

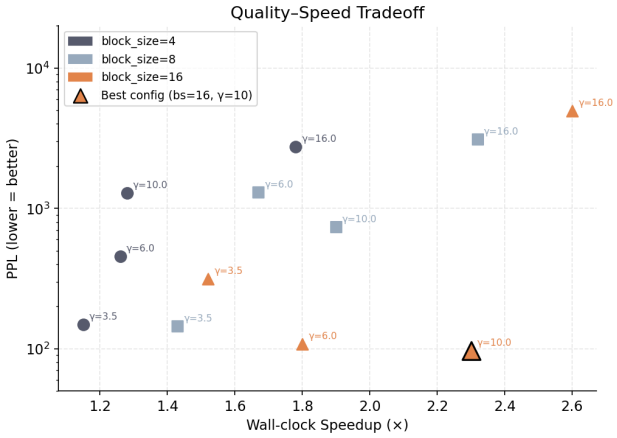


Figure 15. Quality-speed Tradeoff

Effect of block size. Larger block sizes consistently yield higher wall-clock speedup under the same γ. The configuration $k = 16$, $\gamma = 10.0$ achieves a wall-clock speedup of 2.30× with a PPL of 96.5, the best quality-speed tradeoff among all SBD settings. This suggests that a larger block provides richer bidirectional attention context within the block, allowing the model to make more accurate predictions once some tokens are revealed.

Effect of γ. NFE speedup increases monotonically with γ, but PPL degrades rapidly at high values. $\gamma = 3.5$ provides the best quality preservation while still achieving meaningful acceleration.

5.6 Conclusion/Discussion

We implemented and evaluated SBD across both training and inference in GPT-2-small setting. On the training side, we fine-tuned GPT-2 with both the autoregressive loss and the masked parallel prediction loss. The training curves show that SBD can learn the masked prediction objective while preserving similar autoregressive training behavior to the baseline. On the inference side, we built a MiniTorch implementation of SBD decoding, including the SBD attention mask, entropy-bounded sampling, and KV-cache support.

The main takeaway is that SBD can reduce forward evaluations and also bring real inference speedup. In our best setting, with block size 16 and gamma 10, SBD achieves 2.91x NFE speedup and 2.30x wall-clock speedup, even with system overhead. Compared with a more optimized PyTorch or CUDA-based implementation, our MiniTorch implementation has extra overhead from sources such as Python control flow, MiniTorch tensor operations, and KV-cache concatenation. These costs likely explain part of the gap between NFE speedup and wall-clock speedup.

Another limitation is generation quality. Although SBD

gives clear speedup, its PPL is still much higher than the NTP baseline in our experiments. In the best setting discussed above, SBD has a PPL of 96.5, compared with 1.67 for the NTP baseline. We also use much larger gamma values than those reported in the paper. This may be related to our use of GPT-2 small and a smaller training setup, which could make the model less confident when predicting multiple future tokens in parallel.

For future work, we would like to address both limitations. On the system side, customized CUDA kernels for KV-cache updates could reduce overhead and make wall-clock speedup closer to NFE speedup. On the model side, scaling to larger datasets, training for more steps, and testing larger models would make the evaluation more comparable to the paper and help us better study the speed-quality tradeoff.

5.7 Team Member Contributions

Both team members contributed equally to this project. The MiniTorch part, dataset and model selection, benchmark evaluation, and poster preparation were carried out collaboratively.

Yuhang Hu led the implementation of the SBD training pipeline, including the joint NTP and MATP loss function, the hybrid attention mask for training, and the weight conversion, etc.

Miaoyang Ye led the implementation of the data loading pipeline, the SBD inference attention mask, and the KV-cache optimization in MiniTorch, including the numpy-based K/V concatenation, etc.

REFERENCES

Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. Medusa: Simple llm inference acceleration framework with multiple decoding heads, 2024. URL https://arxiv.org/abs/2401.10774.

Gat, I., Ben-Hamu, H., Havasi, M., Haziza, D., Reizenstein, J., Synnaeve, G., Lopez-Paz, D., Karrer, B., and Lipman, Y. Set block decoding is a language model inference accelerator, 2025. URL https://arxiv.org/abs/2509.04185.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding, 2023. URL https://arxiv.org/abs/2211.17192.

Stern, M., Shazeer, N., and Uszkoreit, J. Blockwise parallel decoding for deep autoregressive models, 2018. URL https://arxiv.org/abs/1811.03115.